



NED UNIVERSITY OF ENGINEERING & TECHNOLOGY
KARACHI

CANTEEN MANAGEMENT SYSTEM

An Object-Oriented Design for Role-Based Ordering, Discounting, and Stock Management

Complex Computing Problem (CCP) Technical Report

Prepared By:

Hasan Shahir (CT-25095)

Bushra Abid (CT-25057)

Areeba (CT-25070)

Eshba (CT-25052)

Hassaan Hameed (CT-25097)

Object-Oriented Programming (CT-260) | Summer 2026
NED University of Engineering & Technology, Karachi

1. Problem Statement

University and office canteens typically rely on manual, cash-register-style order taking: a cashier notes down items on paper or a simple till, tracks remaining stock by memory or a whiteboard, and applies discounts inconsistently across customer types. This approach is slow during rush hours, offers no audit trail of what was sold or when stock runs low, and cannot reliably tell a paying student apart from a staff member entitled to a concession.

This project addresses that gap with an object-oriented Canteen Management System that (a) models the menu as a hierarchy of item types, (b) tracks per-item stock independently of the menu catalogue, (c) verifies a customer's role from their ID without storing any password or credential, and (d) applies the correct discount policy automatically based on that role. The goal is a small but faithfully-designed system where every one of these responsibilities is owned by a distinct class, so that new item types, new roles, or new discount rules can be added without rewriting existing logic.

2. OOP Principles Identification

2.1 Encapsulation

Definition: Encapsulation bundles data with the methods that operate on it, and restricts direct access to that data through access specifiers (private, protected, public), so that an object's internal state can only change through its own controlled interface.

Implementation: The Stock class keeps totalQuantity and remainingQuantity as private members. External code can never decrement stock directly — it must go through reduceStock(), which checks availability before allowing a sale, or addStock(), which is the only path for replenishing inventory.

```
class Stock {
private:
    string itemId;
    int totalQuantity;
    int remainingQuantity;
public:
    bool reduceStock(int qty = 1) {
        if (remainingQuantity >= qty) {
            remainingQuantity -= qty;
            return true;
        }
        return false;
    }
    void addStock(int qty) { totalQuantity += qty; remainingQuantity += qty; }
};
```

Because remainingQuantity can never go negative and totalQuantity can never be reduced by accident, the class guarantees its own consistency regardless of how carelessly the calling code behaves.

2.2 Abstraction

Definition: Abstraction exposes only what a caller needs to know and hides how the result is produced. Both MenuItem and Person declare pure virtual methods, forcing every concrete subclass to supply its own implementation while callers only ever depend on the abstract interface.

```
class MenuItem {
protected:
    string itemId; string name; double price;
public:
    virtual void display() = 0;    // how it prints is hidden
};
```

```

    virtual ~MenuItem() {}
};

class Person {
protected:
    string name; string id;
public:
    virtual double applydiscount(double subtotal) = 0;
    virtual string getrole() = 0;
};

```

The billing code in `printBillAndOrder()` calls `customer.applydiscount(subtotal)` without ever knowing, or needing to know, whether the underlying object is a `Student` or an `Employee`.

2.3 Inheritance

Definition: Inheritance lets a subclass reuse the attributes and behaviour of a base class while adding or overriding only what is specific to it, avoiding duplicated code across related types.

`Food` and `Drink` both derive from `MenuItem`, inheriting `itemId`, `name` and `price` along with `getId()`, `getName()`, and `getPrice()`; `Food` alone adds a `category` field. Likewise, `Student` and `Employee` derive from `Person`, inheriting `name` and `id` and the constructor that sets them, while each supplies its own discount rule and role label.

2.4 Polymorphism

Definition: Polymorphism allows a base-class pointer or reference to invoke a subclass's overridden implementation at runtime, so the same call site behaves differently depending on the object's actual type.

```

void printBillAndOrder(Menu &menu, Person &customer, const string &custName) {
    cout << "Welcome " << custName << " (" << customer.getrole() << ")" << endl;
    ...
    double finalTotal = customer.applydiscount(subtotal);
}

```

Here `customer` is declared as a plain `Person&`, yet `applydiscount()` and `getrole()` dispatch to `Student`'s or `Employee`'s own version depending on which object was actually passed in from `placeOrder()`. The same pattern appears in `Menu::displayAll()`, where `items[i]->display()` prints a `Food` or a `Drink` correctly through a single loop over `MenuItem*` pointers.

3. Design Patterns

Beyond the four core OOP principles, the system deliberately applies two well-known design patterns — one creational, one behavioural — to keep object construction and discount logic from turning into scattered conditional code.

3.1 Factory Pattern (Creational)

The Factory Pattern creates objects without the calling code needing to know, or specify, the exact concrete class being instantiated. It centralises all “object creation” decisions in a single place.

Where it is in the code: the `MenuItemFactory` class.

```

class MenuItemFactory {
public:
    static MenuItem* createItem(string type, string itemId, string name,
                                double price, string category = "") {
        if (type == "Food") return new Food(itemId, name, price, category);
        if (type == "Drink") return new Drink(itemId, name, price);
    }
};

```

```

        return nullptr;
    }
};

// main() never calls `new Food(...)` or `new Drink(...)` directly:
menu.addItem(MenuItemFactory::createItem("Food", "F01", "Chicken Biryani", 250, "Meal"),
10);

```

How it works: rather than `main()` calling `new Food(...)` or `new Drink(...)` directly, it calls `MenuItemFactory::createItem(...)` and passes a type string such as "Food" or "Drink". The factory reads that string and decides which concrete subclass to instantiate, handing the result back as a generic `MenuItem*` pointer.

Why it is good: if a new category such as `Dessert` is introduced later, there is exactly one place to update — the factory — instead of hunting down every call site in `main()` that constructs a menu item.

3.2 Strategy Pattern (Behavioural)

The Strategy Pattern defines a family of interchangeable algorithms and encapsulates each one inside its own object, so the algorithm can vary independently of the code that uses it.

Where it is in the code: the `Person` / `Student` / `Employee` hierarchy, specifically `applydiscount(double subtotal)`.

```

class Student : public Person {
public:
    double applydiscount(double subtotal) override { return subtotal; }
    string getrole() override { return "Student"; }
};

class Employee : public Person {
public:
    double applydiscount(double subtotal) override {
        return subtotal - subtotal * 0.05;
    }
    string getrole() override { return "Employee"; }
};

// Billing code stays completely branch-free:
double finalTotal = customer.applydiscount(subtotal);

```

How it works: computing the final bill requires a discount algorithm, but instead of writing a block of if-else statements in the billing function (e.g. checking whether the customer is a student or an employee), each discount “strategy” is encapsulated directly inside the `Student` or `Employee` object itself. `printBillAndOrder()` simply calls `customer.applydiscount(subtotal)` and lets that object's own strategy perform the calculation.

Why it is good: it removes branching logic from the billing function entirely and follows the Open/Closed Principle — the system is open to extension but closed to modification. Adding a `Faculty` class with a 15% discount later means writing that one class and giving it its own `applydiscount()` logic; `printBillAndOrder()` does not need to be touched at all.

4. Code Architecture

The system is organised into two independent hierarchies — `MenuItem` (catalogue) and `Person` (customer roles) — plus two coordinating classes, `Stock` and `Menu`, and a pair of free functions that glue ordering to billing. The complete class diagram is shown below.

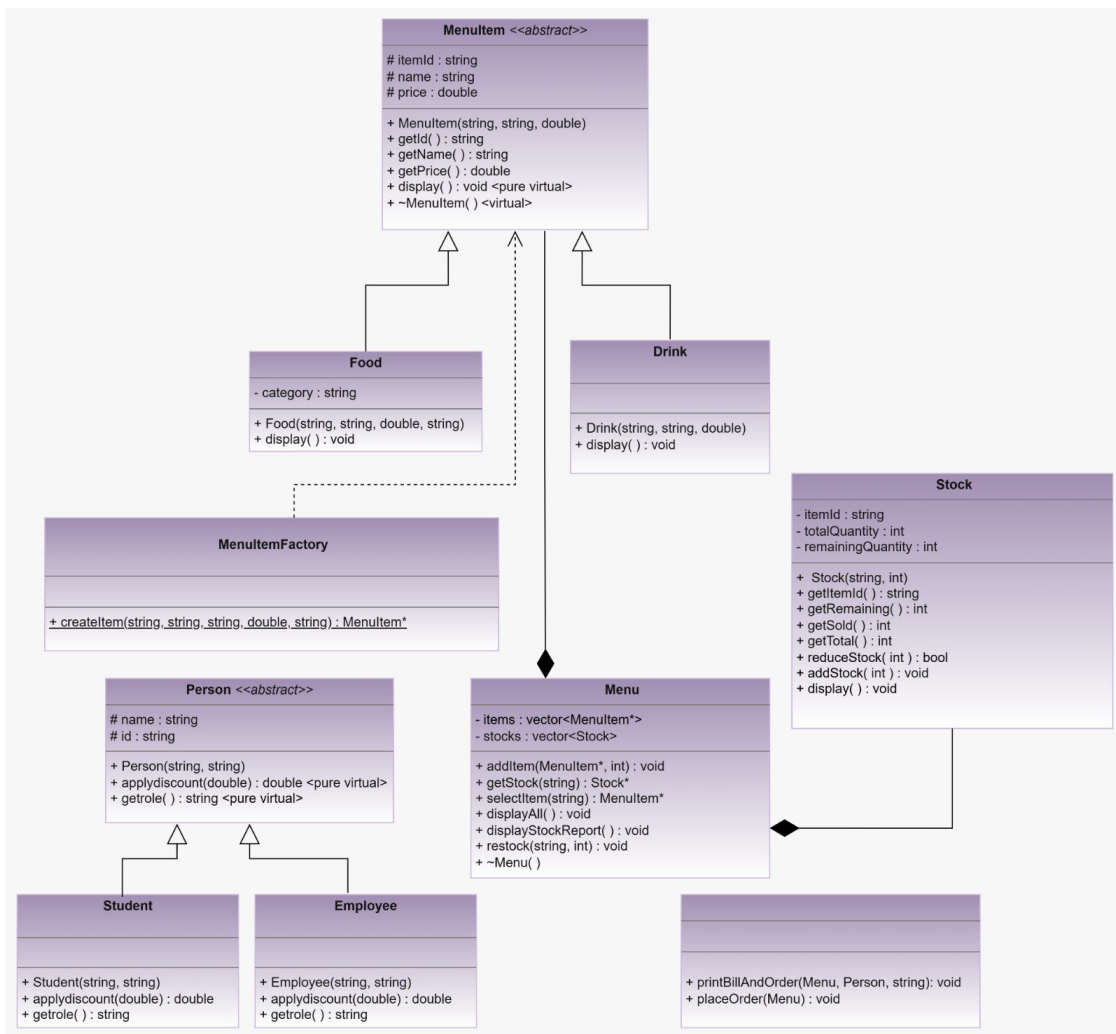


Figure 1. Class diagram of the Canteen Management System — MenuItem/Person hierarchies, Stock, Menu, and MenuItemFactory.

3.1 MenuItem Hierarchy and the Factory

MenuItem is an abstract base class holding the fields common to every sellable item (itemId, name, price) and declaring display() as pure virtual. Food adds a category attribute (e.g. "Fast Food", "Snack"); Drink needs no extra field and simply overrides display(). MenuItemFactory::createItem() is the single point where a type string is translated into the correct concrete object, returned upward as a MenuItem*.

3.2 Stock — Inventory Tracking Decoupled from the Catalogue

Rather than storing a quantity field inside MenuItem itself, stock is modelled as a separate Stock object keyed by itemId. This keeps the catalogue (what an item is) independent of its inventory (how many are left), so the same MenuItem could in principle be reused across multiple stock pools without duplicating item metadata. Menu holds parallel vectors of MenuItem* and Stock, and getStock(itemId) / selectItem(itemId) look each up by matching ID.

3.3 Person Hierarchy — Role Verification Without Credential Storage

Person stores only a name and an id — no password or PIN is ever stored, so there is nothing sensitive to leak or protect. placeOrder() reads the customer's typed ID and inspects its first three characters: an ID beginning with "std" is treated as a Student, one beginning with "emp" as an Employee, and anything else throws a runtime_error("Invalid ID") that is caught and reported without crashing the program. This keeps identity verification lightweight and ID-based rather than credential-based, matching the project's design constraint of not persisting any authentication secrets.

```
if (id.substr(0, 3) == "std")      { Student s(name, id); printBillAndOrder(menu, s,
name); }
else if (id.substr(0, 3) == "emp") { Employee e(name, id); printBillAndOrder(menu, e,
name); }
else throw runtime_error("Invalid ID");
```

3.4 Menu — Aggregating Items and Stock

Menu owns the full catalogue and inventory: addItem() registers a new MenuItem alongside its starting Stock, displayAll() and displayStockReport() iterate over both in lock-step using dynamic_cast to separate Food from Drink for reporting, and restock() forwards a quantity increase to the matching Stock object. Its destructor deletes every owned MenuItem*, since Menu is the sole owner of that memory.

3.5 printBillAndOrder() and placeOrder() — Tying the Hierarchies Together

printBillAndOrder() is the function that actually connects the two hierarchies: it takes a Menu& and a Person& together, runs the item-selection loop against the stock-aware menu, accumulates a subtotal, and then calls customer.applydiscount(subtotal) polymorphically to compute the final total. placeOrder() is the entry point that reads the raw name/ID pair, decides — via the ID prefix check above — which concrete Person subclass to construct, and passes it into printBillAndOrder() by reference, wrapping the whole flow in a try/catch so an invalid ID produces a clean error message instead of undefined behaviour.

5. Limitations and Future Work

4.1 No Persistent Storage

Problem: All menu items, stock levels and orders live only in in-memory vectors. Every count — items sold, stock remaining, revenue — is lost the moment the program exits.

```
// Current: in-memory only
vector<MenuItem*> items;
vector<Stock> stocks;
```

Proposed: Serialize the menu and stock state to a file (JSON or a simple delimited format) on exit and reload it on startup, or back the Menu class with a lightweight embedded database such as SQLite so stock and sales survive restarts.

4.2 Prefix-Based Role Verification

Problem: A role is decided purely from the first three characters of a self-typed ID string. Anyone can type "emp999" and receive the employee discount — there is no check against an actual roster of registered students or staff.

```
// Current: trusts the prefix the user types if (id.substr(0, 3) == "emp") { Employee
e(name, id); ... }
```

Proposed: Look the ID up against a pre-registered list (a file or database of valid student/employee IDs) so the role is confirmed rather than assumed, while still storing no password — preserving the "no credential storage" design constraint but closing the spoofing gap.

4.3 Hardcoded, Two-Tier Discount Model

Problem: Employee::applydiscount() hardcodes a flat 5% and Student::applydiscount() hardcodes 0%. Adding a third role (e.g. Faculty, Senior Citizen, or a promo-code discount) requires editing and recompiling the class hierarchy.

```
// Current: fixed at compile time double applydiscount(double subtotal) override { return
```

```

subtotal - subtotal * 0.05; }

// Proposed: discount rate supplied externally class Employee : public Person
{
    double rate; public:      Employee(string n, string i, double r) : Person(n, i),
    rate(r) {}               double applydiscount(double subtotal) override { return subtotal * (1 -
    rate); } };

```

Proposed: Load discount rates from a configuration file and consider a Strategy-pattern DiscountPolicy object composed into Person, so new discount rules can be added without touching Student or Employee at all.

4.4 Console-Only, Single-Order Interface

Problem: The system is entirely console-driven with cin >> reads; there is no receipt export, no history of past orders, and no way to review a previous customer's bill once it scrolls off screen.

Proposed: Log every completed order (customer, items, subtotal, discount, total, timestamp) to a running order-history file or table, and expose the same Menu/Stock/Person logic behind a simple GUI or web front end so the underlying classes would not need to change at all — only a new presentation layer would be added.

6. Testing and Sample Execution

Because this project is a role-based transactional system rather than a search algorithm, correctness was verified through targeted functional test cases covering both hierarchies (item selection/stock and person/discount) and the exception path, instead of a performance benchmark.

#	Test Case	Input / Action	Expected Behaviour	Result
1	Student order	ID "std101", order F01 + D01	No discount applied; total = subtotal	Pass
2	Employee order	ID "emp204", order F02 + D01	5% discount applied to subtotal	Pass
3	Invalid ID prefix	ID "abc123"	runtime_error("Invalid ID") caught, error message shown, program continues	Pass
4	Stock depletion	Order an item after remaining reaches 0	"Sorry, <item> is out of stock!" and item excluded from order	Pass
5	Restock flow	Restock an existing item ID by qty	Stock's totalQuantity and remainingQuantity both increase; "Stock updated!" printed	Pass
6	Restock invalid item	Restock a non-existent item ID	"Item not found!" printed, no state changed	Pass

7. Team Contribution

Name	Roll No.	Contribution
Hasan Shahir	CT-25095	Team lead; report structuring and documentation; UML class diagram design and coordination across modules
Bushra Abid	CT-25057	Vice team lead; implementation of the Person class hierarchy (Student, Employee) and ID-based role verification logic
Areeba	CT-25070	Designed the system's UML model; implemented the MenuItem, Food, and Drink classes; built MenuItemFactory using the Factory design pattern

Name	Roll No.	Contribution
Eshba	CT-25052	Design and implementation of the Stock class and restocking functionality
Hassaan Hameed	CT-25097	Design and implementation of discount-related classes and logic

8. References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
2. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
3. Deitel, P., & Deitel, H. (2016). C++ How to Program (10th ed.). Pearson.
4. Martin, R. C. (2003). Agile Software Development: Principles, Patterns, and Practices. Prentice Hall.
5. ISO/IEC. (2020). ISO/IEC 14882:2020 — Programming Languages — C++.